

Assignment No.

02

Combinational Logic Design

FAROOQ ABDULLA

Online Internship on Analog and Digital VLSI Design Flow

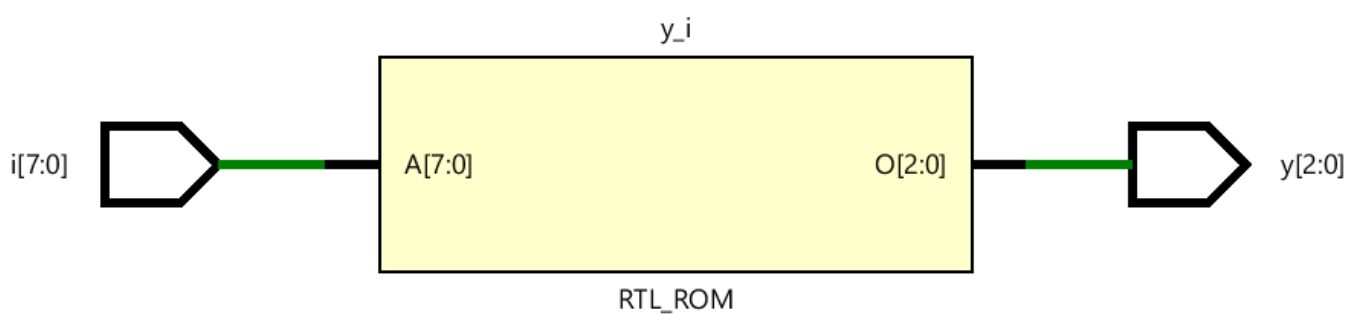
Batch 01

Q1. Design and Simulate 8:3 Encoder. Simulate all possible input combinations and verify the output.

CODE:

```
module encoder_8to3 (  
    input [7:0] i,  
    output reg [2:0] y);  
  
    always @(*) begin  
        case (i)  
            8'b00000001: y = 3'b000;  
            8'b00000010: y = 3'b001;  
            8'b00000100: y = 3'b010;  
            8'b00001000: y = 3'b011;  
            8'b00010000: y = 3'b100;  
            8'b00100000: y = 3'b101;  
            8'b01000000: y = 3'b110;  
            8'b10000000: y = 3'b111;  
            default: y = 3'b000;  
        endcase  
    end  
  
endmodule
```

SCHEMATIC:



TESTBENCH CODE:

```
module tb_encoder_8to3;
    reg [7:0] i;
    wire [2:0] y;

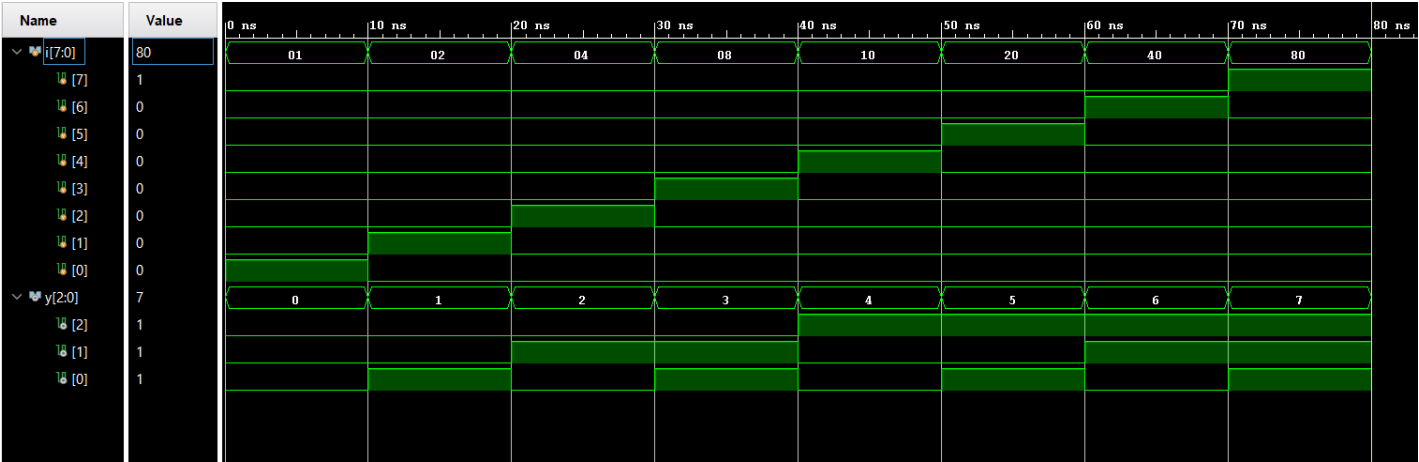
    encoder_8to3 uut (
        .i(i),
        .y(y));

    initial begin
        $display("Time\t Input (I) \t Output (Y)");
        $monitor("%0t\t %b \t %b", $time, i, y);

        i = 8'b00000001; #10;
        i = 8'b00000010; #10;
        i = 8'b00000100; #10;
        i = 8'b00001000; #10;
        i = 8'b00010000; #10;
        i = 8'b00100000; #10;
        i = 8'b01000000; #10;
        i = 8'b10000000; #10;

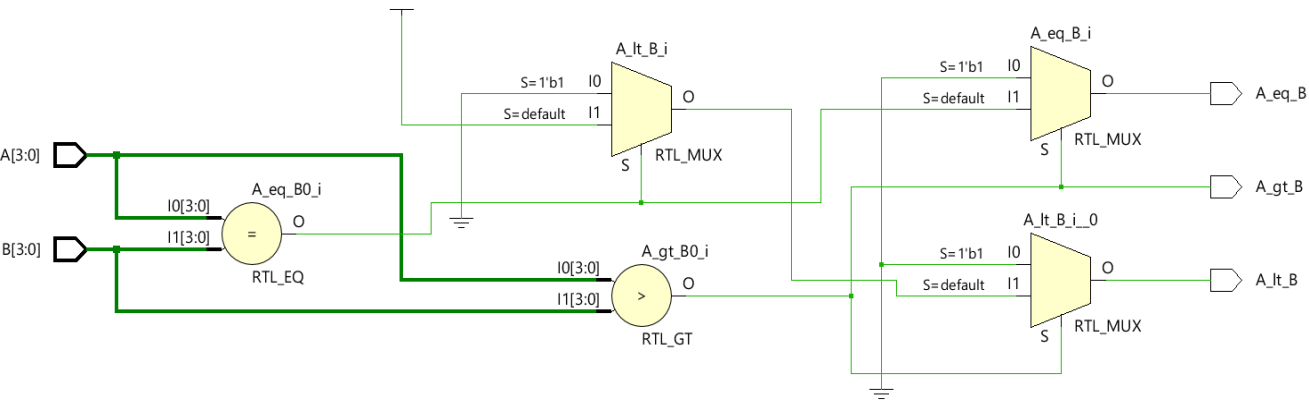
        $finish;
    end
endmodule
```

SIMULATION:



- $A > B$
- $A = B$
- $A < B$

SCHEMATIC:

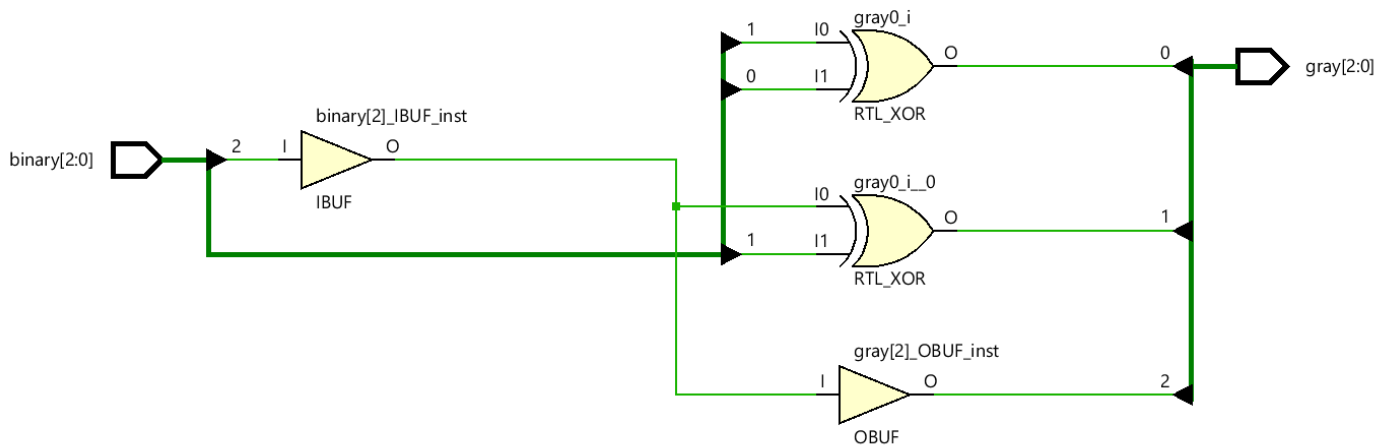


Q3. Design and simulate 3-bit Binary-to-Gray code converter and verify its operation through simulation.

CODE:

```
module binary_to_gray_3bit (  
    input [2:0] binary,  
    output [2:0] gray);  
  
    assign gray[2] = binary[2];  
    assign gray[1] = binary[2] ^ binary[1];  
    assign gray[0] = binary[1] ^ binary[0];  
  
endmodule
```

SCHEMATIC:



TESTBENCH CODE:

```
module tb_binary_to_gray_3bit;
    reg [2:0] binary;
    wire [2:0] gray;

    integer i;

    binary_to_gray_3bit uut (.binary(binary),.gray(gray));

    initial begin
        $display("Time\t Binary \t Gray");
        $monitor("%0t\t %b \t %b", $time, binary, gray);

        for (i = 0; i < 8; i = i + 1) begin
            binary = i;
            #10;
        end

        $finish;
    end
endmodule
```

SIMULATION:

